

Relatório lab 05

Neste laboratório foi requisitado a criação de programas em Python que desempenhassem o papel de servidor e cliente para acesso por sockets UDP para enviar um texto do cliente para o servidor em UTF-8 e o servidor enviar o texto codificado em base64 para ser mostrado na tela do cliente. Antes de partir para o exercício fornecido, segui as instruções apontadas no roteiro para criar um servidor e cliente básicos para se conectarem a partir da utilização de conexões UDP. Segue o código em Python utilizado para o servidor e para o cliente, respectivamente:

Servidor:

```
from socket import *

portaServidor = 12345
socketServer = socket(AF_INET, SOCK_DGRAM)
socketServer.bind(("", portaServidor))
print("O servidor está pronto para receber requests")

while(True):
    mensagem, enderecoServer = socketServer.recvfrom(2048)
    msgMod = mensagem.decode().upper()
    socketServer.sendto(msgMod.encode(), enderecoServer)
```

Cliente:

```
from socket import *
nomeServer = "localhost"
portaServer = 12345

socketCliente = socket(AF_INET, SOCK_DGRAM)
mensagem = input("Insira uma mensagem em letras minúsculas: ")
socketCliente.sendto(mensagem.encode(), (nomeServer, portaServer))
msgMod, serverAddress = socketCliente.recvfrom(2048)
print(msgMod.decode())

socketCliente.close()
```

Vou explicar um pouco de como esses códigos funcionam. A primeira linha importa todas as funções contidas na biblioteca 'socket'. Essa biblioteca é uma interface de baixo nível e provém acesso à interface de Sockets de Berkeley (BSD), uma API utilizada para o compartilhamento de dados pela comunicação entre processos sendo executados em um sistema de computador (Inter-Process Communication, IDP). A palavra 'socket' é uma palavra abstrata que representa o local final de um caminho de comunicação de rede, e os BSDs representam, no sistema Unix, um descritor de arquivo que provém uma interface comum para entrada e saída de dados. Essa biblioteca possui funções que permitem criar sockets personalizados para a transferência de dados através da rede.

A função utilizada no código permite criar sockets TCP/IP e UDP para transferência de dados, ela requer dois parâmetros, o primeiro representa o tipo de endereço que será

utilizado e está organizada em famílias, por exemplo, o utilizado no código é utilizado para protocolos UDP, que é orientado a datagramas, o que significa que ele provém ou consome mensagens curtas e limitadas, porém existem outros tipos para diferentes propósitos, como o INET para conexões TCP e Bluetooth para conexões bluetooth.

Para permitir o acesso a clientes, o servidor precisa criar uma porta que serve como uma chave para apontar o processo que o servidor está utilizando. Como o servidor está sendo executado na rede local, foi necessário fornecer um endereço da rede local no código do cliente para ele conseguir se conectar, caso o servidor não utilize o endereço que forneceu, ele não vai conseguir achar a sua mensagem e a mensagem será perdida, que foi o caso da minha primeira tentativa. Em vez de colocar o endereço como “localhost”, havia deixado “hostname”, que não é um endereço válido, devolvendo o primeiro erro abaixo:

```
socket.gaierror: [Errno -3] Temporary failure in name resolution
Gui@Ubuntu:~/Documents/redes/lab05$ python UDPClient.py
Input lowercase sentence: hello
Traceback (most recent call last):
  File "/home/Gui/Documents/redes/lab05/UDPClient.py", line 7, in <module>
    clientSocket.sendto(message.encode(), (serverName, serverPort))
socket.gaierror: [Errno -3] Temporary failure in name resolution
```

A função ‘bind’ serve para associar um socket com um endereço IP e uma porta para ser utilizado na transferência de dados. Vale notar que o servidor não necessita ouvir de nenhum endereço específico, então o endereço no socket do servidor foi deixado em branco para poder captar a mensagem de qualquer endereço naquela porta. Em seguida, o servidor fica em um loop infinito para receber conexões do socket através da função `recvfrom`, com um tamanho de bytes como parâmetro, que no caso foi 2048 como um número arbitrário, decodificar, modificar, codificar e enviar a mensagem modificada para o cliente pela função ‘sendto’, com a mensagem codificada e o socket do cliente.

Já no lado do cliente, ele pede a entrada de uma mensagem para ser modificada e envia ela ao socket do servidor com a função ‘sendto’. Em seguida recebe a mensagem modificada juntamente ao endereço do servidor e imprime a mensagem decodificada, e em seguida encerra o socket aberto no programa. Aqui está o resultado fornecido pelos programas entre o cliente e o servidor.

```
Gui@Ubuntu:~/Documents/redes/lab05$ python UDPClient.py
Insira uma mensagem em letras minúsculas: hello
HELLO
```

A mensagem foi enviada, processada pelo servidor e sua resposta foi recebida com sucesso, obtendo a mensagem ‘hello’ em letras maiúsculas no programa do cliente.

The image shows a Wireshark packet capture of a UDP communication. The packet list shows two packets: a client request and a server response. The details pane for the selected packet (No. 6) shows the following structure:

- Frame 5: 47 bytes on wire (376 bits), 47 bytes captured (376 bits) on interface lo, id 0
- Ethernet II, Src: 00:00:00:00:00:00 (00:00:00:00:00:00), Dst: 00:00:00:00:00:00 (00:00:00:00:00:00)
- Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
- User Datagram Protocol, Src Port: 60172, Dst Port: 12345
- Data (5 bytes)
- Data: 68656c6c6f (Length: 5)

The packet bytes pane shows the raw data in hexadecimal and ASCII. The client's data is 'hello' (hex 68 65 6c 6c 6f) and the server's response is 'HELLO' (hex 48 45 4c 4c 4f).

```
Gui@Ubuntu:~$ netstat -tunap | grep 12345
(Not all processes could be identified, non-owned process info
 will not be shown, you would have to be root to see it all.)
udp        0      0 0.0.0.0:12345        0.0.0.0:*           30044/python
Gui@Ubuntu:~$
```

É possível ver também que os pacotes do cliente e servidor foram captados pelo Wireshark, que utilizaram o endereço local na porta 12345, que é a escolhida para o servidor. O primeiro é o pacote enviado do cliente para o servidor, e o segundo é a resposta do servidor para o cliente. E usando o netstat, é possível verificar que enquanto o servidor está sendo executado, o processo utilizado no socket do servidor fica aberto.

```
Gui@Ubuntu:~$ netstat -tunap | grep 12345
(Not all processes could be identified, non-owned process info
 will not be shown, you would have to be root to see it all.)
Gui@Ubuntu:~$
```

```
Gui@Ubuntu:~/Documents/redes/lab05/teste$ python UDPClient.py
Insira uma mensagem em letras minúsculas: hello
```

Se o cliente requisitar uma resposta do servidor enquanto o mesmo não esteja sendo executado, que se pode verificar que não está sendo executado devido à ausência do socket no comando netstat, o cliente fica procurando uma resposta infinitamente pois não sabe se o servidor recebeu a mensagem.

Agora que está finalizado os programas de teste, começar o exercício. Utilizei o teste como base para resolvê-lo. Aqui os códigos do servidor e do cliente:

Servidor:

```
from socket import *
import base64

portaServidor = 12345
socketServer = socket(AF_INET, SOCK_DGRAM)
socketServer.bind(("", portaServidor))
print("O servidor está pronto para receber requests")

while(True):
    mensagem, enderecoServer = socketServer.recvfrom(2048)
    b = base64.b64encode(bytes(mensagem.decode(), 'utf-8'))
    msgMod = b.decode('utf-8')
    socketServer.sendto(msgMod.encode(), enderecoServer)
```

Cliente:

```
from socket import *
import base64
nomeServer = "localhost"
portaServer = 12345
```

```

socketCliente = socket(AF_INET, SOCK_DGRAM)
mensagem = input("Insira um texto em UTF-8: ")
socketCliente.sendto(mensagem.encode('utf-8'), (nomeServer, portaServer))
print("Mensagem enviada:\t", mensagem)

msgMod, serverAddress = socketCliente.recvfrom(2048)
print("\nMensagem recebida em base64:\t", msgMod.decode())

sampleBytes = base64.b64decode(msgMod)
msgDeco = sampleBytes.decode('utf-8')
print("Mensagem decodificada para UTF-8: ", msgDeco)

socketCliente.close()

```

Para esse exercício, foi necessário importar a biblioteca 'base64', que possui funções para codificação e decodificação de dados para a codificação base64. Mas o que é a codificação base64? É uma codificação de dados binários que foi desenvolvida para ter apenas 64 caracteres no máximo, utilizando apenas 8 bits para armazenar os valores, indo de 'A – Z', 'a – z', '1 – 9' junto com '+' e '/', concluindo a sua capacidade máxima. Para a conversão de um dado para a base64, expande-se os bits e vai dividindo eles em caracteres de 8 bits.

No código do servidor, após ter obtido a mensagem em UTF-8, o código codifica a mensagem em base64 a partir da função 'b64encode' que requer um dado binário para ser convertido. Para obter esse dado, usamos a função 'bytes' do python para transformar a mensagem acompanhado do formato original em um dado binário que o 'b64encode' pode utilizar.

Para enviar isso de volta ao cliente, transformamos o dado em binário para uma string de caracteres com a função 'decode' do python e enviamos isso para o cliente. Apesar de estarmos decodificando o dado, a forma dos bytes se mantém, por isso o dado continua formatado em base64, a única diferença é que se transforma em uma string, em vez de um dado em binário.

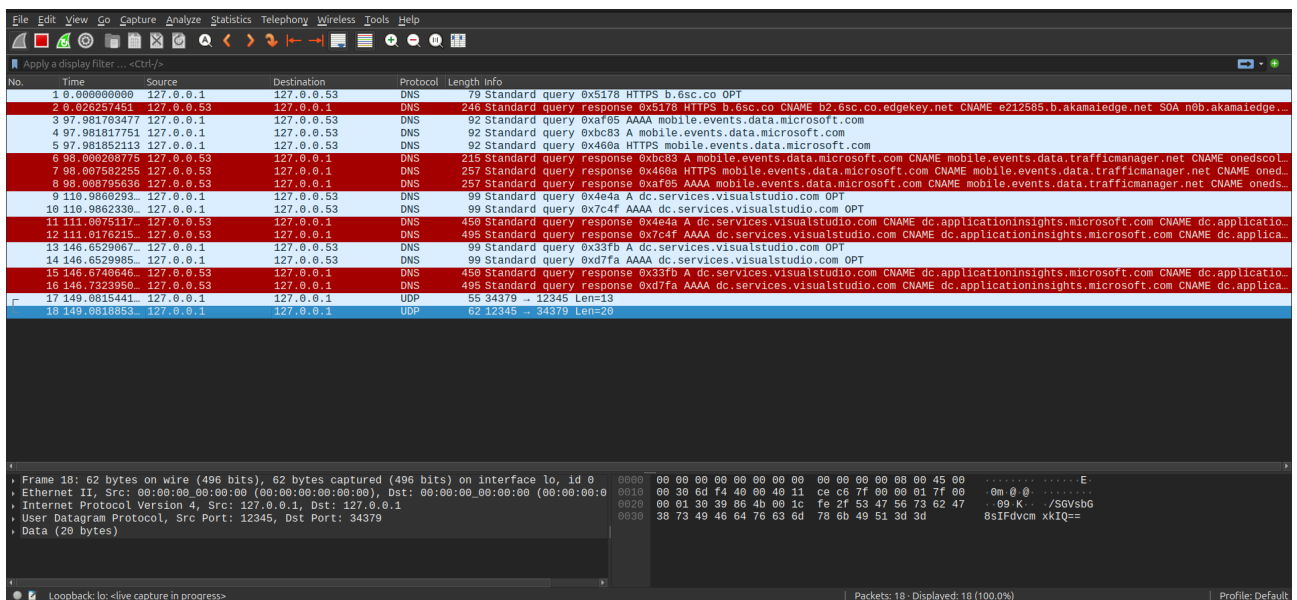
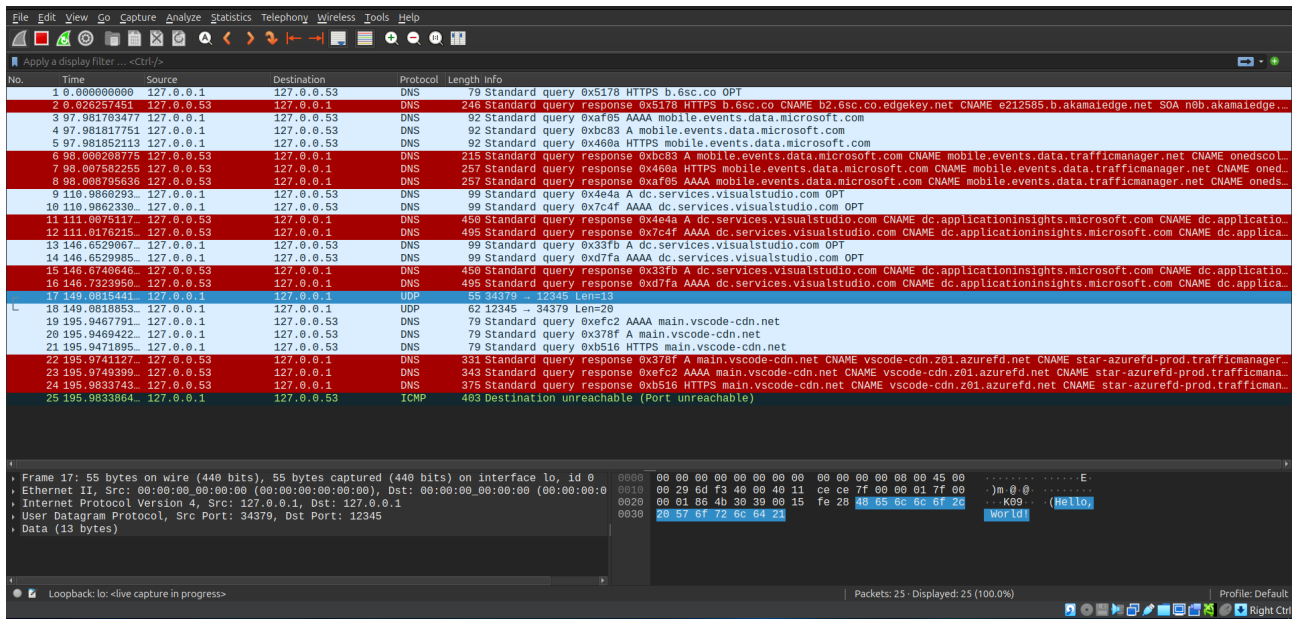
Já no lado do cliente, enviamos a mensagem em UTF-8 e depois recebemos a mensagem modificada do servidor para ser mostrada ao cliente. Como um extra, resolvi pegar essa mensagem recebida e tentar decodificar de volta para o UTF-8 sem usar a mensagem enviada, apenas a recebida. Foi necessário usar a função 'b64decode' que funciona da forma oposta do 'b64encode'. Isso decodifica os bytes de volta para a formatação original, que mostra a mensagem original.

```

Gui@Ubuntu:~/Documents/redes/lab05/exercicio$ python UDPClient.py
Insira um texto em UTF-8: Hello, World!
Mensagem enviada:      Hello, World!

Mensagem recebida em base64:      SGVsbG8sIFdvcmxkIQ==
Mensagem decodificada para UTF-8: Hello, World!
Gui@Ubuntu:~/Documents/redes/lab05/exercicio$

```



Foi realizada a captura dos pacotes enviados no Wireshark, junto com o conteúdo enviado. Na primeira imagem é possível identificar que o pacote foi enviado do cliente para o servidor, e não possui nenhuma codificação, enquanto o segundo é do servidor para o cliente, e envia a mensagem original codificada em base64, o que faz a leitura da mensagem para clientes externos ficar mais complicada.

Com isso, conclui-se a resolução do exercício do laboratório 05. Os programas foram executados com sucesso, utilizando o protocolo UDP para obter e enviar informações pela internet.